Prosit 1 : ARCHIE

Rôles :

- Animateur : Enzo
- Scribe : Thaïs
- Secrétaire : Lisa
- Gestionnaire du temps : Romain

1. Contexte

Nous sommes quatre collaborateurs de l'entreprise et nous venons de reprendre le projet CHU (Cloud Healthcare Unit), alors qu'il ne reste que trois semaines avant l'échéance. Archie, un ancien stagiaire, avait lancé le projet : il a fait la veille technologique et l'étude de faisabilité, dessiné une architecture Big Data complète et monté un environnement virtualisé avec les outils nécessaires. Le projet a reçu un « go », mais il est resté en stand-by à la fin de son stage, faute de budget pour des ressources externes.

Le groupe CHU, qui sera l'utilisateur final, attend une maquette qui exploite ses données selon le cahier des charges. Notre manager, lui, doute que deux jours suffisent pour choisir l'architecture. Plusieurs difficultés se présentent. L'architecture proposée par Archie est complexe et on voit mal comment la relier à l'environnement virtualisé. Les données brutes sont stockées dans « Mes documents » et ne sont pas exploitables en l'état, à cause de leur structure et de problèmes de performances. Ce sont aussi des données de santé très personnelles, ce qui impose des exigences fortes de confidentialité et de sécurité.

2. Mots-clés

Maquette	Prototype fonctionnel qui démontre la faisabilité et la valeur de la solution. Ce n'est pas un produit fini.
Big Data / 5V	Données trop volumineuses ou trop variées pour les outils classiques. On les décrit par 5V : Volume, Vélocité, Variété, Véracité, Valeur.
Architecture Big Data	Chaîne qui va des sources à l'ingestion, au stockage (data lake), au traitement (batch ou streaming), à la mise à disposition (entrepôt, data marts) puis à la restitution (BI).
Base multidimensionnelle (OLAP)	données organisées en cubes (faits × dimensions) pour l'analyse. S'oppose à l'OLTP, orienté transactions.
Data Warehouse	Base centralisée pour le décisionnel. Elle est orientée sujet, intégrée, historisée et non volatile.
Data Mart	Sous-ensemble de l'entrepôt dédié à un métier ou à un sujet d'analyse.
Data Lake	Stockage brut de données de tout format. Le schéma est appliqué à la lecture (schema-on-read)
Modèle logique	Structure indépendante de la technologie (faits, dimensions, relations)
Modèle physique	Implémentation concrète dans un outil (types, partitions, index, formats de fichiers).
Intégration des données (ETL)	Extraction, puis transformation (nettoyage, normalisation, anonymisat la cible.
Système de fichiers distribué (HDFS)	fichiers découpés en blocs répliqués sur plusieurs nœuds. Le NameNode gère les métadonnées, les DataNodes stockent les blocs.

HDS	certification Hébergeur de Données de Santé, obligatoire en France pour héberger des données de sand.
PowerBI	outil Microsoft de BI et de création de tableaux de bord.
MapReduce	modèle de traitement distribué en 3 étapes (Map, Shuffle, Reduce)
Cloudera	entreprise américaine de logiciels de gestion des données et d'intelligence artificielle

3. Problématique

L'architecture et le travail préparatoire d'Archie sont-ils réellement exploitables en l'état, et comment re-concevoir une solution Big Data/Data Mart conforme au RGPD et performante pour le CHU en 3 semaines ?

4. Contraintes

Explicites :
- Délai : maquette complète sous 3 semaines (14 jours planifiés).
- Budget : aucun budget pour des ressources externes.
- Équipe : 4 personnes, sans expérience dans la mise en place d'une solution Big Data.
- Existant : partir des livrables d'Archie (schéma, environnement virtualisé, données).
- Volume : volume de données important, qui impose une architecture Big Data.
- Performance : les données brutes ne peuvent pas être exploitées directement.

- Livrable : maquette qui répond aux enjeux du CHU et aux analyses du cahier des charges, exploitation et visualisation comprises.

Implicites :
- Règlements de santé:
- Sécurité :
  - données de santé stockées dans « Mes documents », donc non sécurisées ;
  - il faut un contrôle d'accès, une traçabilité et du chiffrement.
- Techniques :
  - outils limités à l'environnement virtualisé existant ou à de l'open source gratuit ;
  - la solution doit pouvoir passer à l'échelle après la maquette.
- Éthiques : responsabilité envers les patients, qui dépasse le seul aspect technique.

5. Plan d’action [17h]

- Chiffrement des données brutes, script d'anonymisation RGPD et tri de la VM d'Archie [2h]
- Conception du MLD (Faits / Dimensions), tests de connexion et validation architecture [3h]
- Développement des scripts d'ingestion/anonymisation et chargement dans le Data Lake [2.5h]
- Création des tables physiques, partitionnement (Parquet) et alimentation Data Warehouse / Data Marts [3h]
- Création des vues OLAP, agrégations et tests de performance [2h]
- Connexion BI, calcul des indicateurs CHU, tableaux de bord et contrôle RGPD [2h]
- Recette fonctionnelle et rédaction de la documentation technique [1h]
- Répétition, préparation et présentation officielle de la maquette au CHU [1.5h]

6. Réalisation

1. Sécurisation des données et assainissement de la VM d'Archie (2 h)

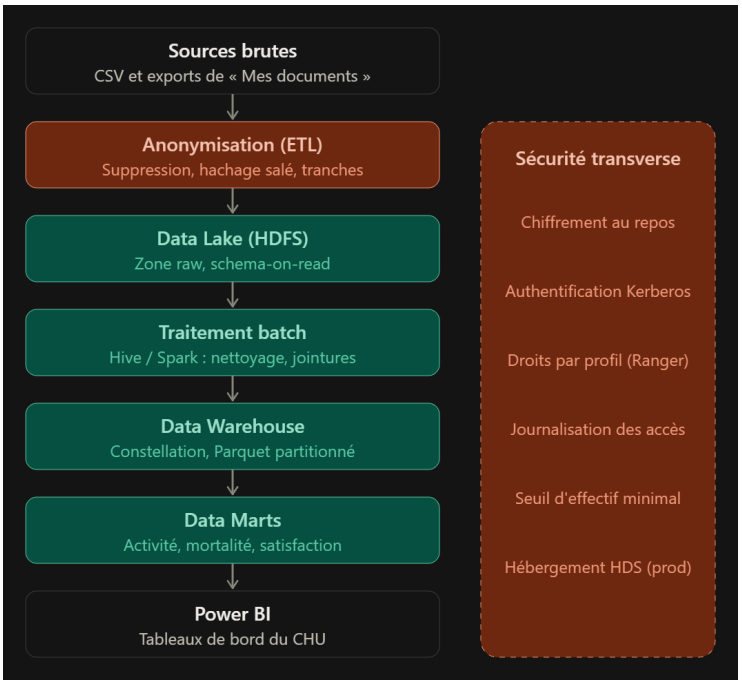
- **Tri et nettoyage de l'environnement :** Audit de la VM fournie par le stagiaire, suppression des fichiers temporaires, scripts obsolètes et doublons afin de stabiliser l'environnement de travail.
- **Chiffrement au repos :** Configuration du chiffrement des volumes de stockage contenant les fichiers bruts reçus pour empêcher tout accès non autorisé au niveau système.
- **Anonymisation et pseudonymisation RGPD :** Écriture et exécution d'un script d'assainissement avant toute ingestion :
    - Suppression des identifiants directs (nom, prénom, numéro de sécurité sociale).
    - Hachage salé des identifiants secondaires (N° patient local) pour permettre le rapprochement sans lever l'anonymat.
    - Perturbation/agrégation des variables sensibles (ex. transformation de la date de naissance exacte en tranche d'âge).

Donnée brute	Traitement appliqué	Donnée en sortie
Nom, prénom	Suppression	—
N° de sécurité sociale	Suppression	—
N° patient local	Hachage salé (SHA-256 + sel secret)	id_patient_hash
Date de naissance	Généralisation	Tranche d'âge (0-17, 18-39, 40-64, 65+)
Adresse complète	Généralisation	Département

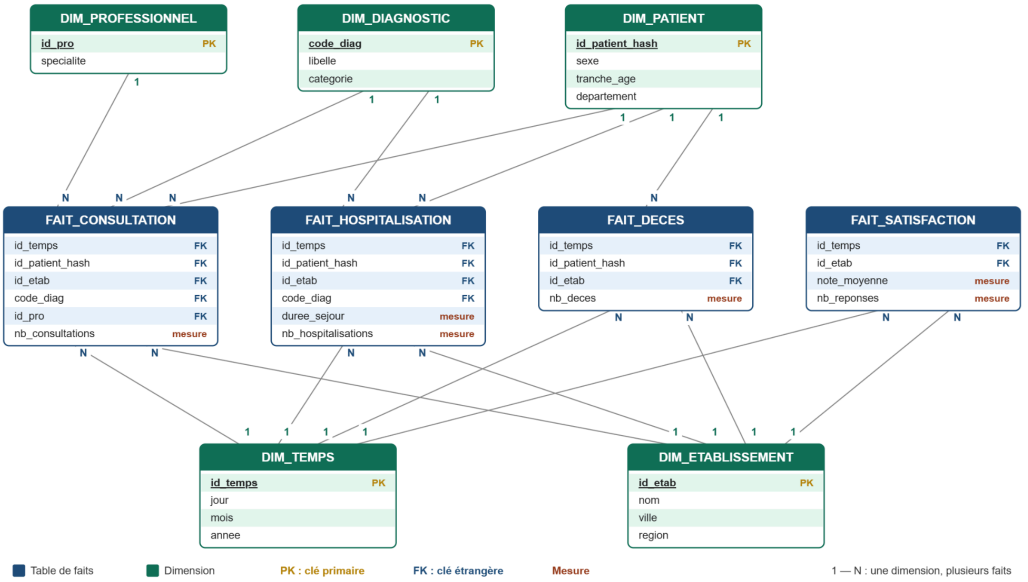
Dates de séjour	Conservées (utiles aux analyses)	Date (dimension Temps)

2. Conception du modèle logique et validation de l'architecture (3 h)

Analyse de l'architecture d'Archie : On confronte le schéma d'Archie à ce qui est réellement installé sur la VM (Cloudera : HDFS, Hive, Impala/Spark). On garde uniquement les briques utiles à la maquette et on écarte celles qui ajoutent de la complexité sans répondre au cahier des charges (par exemple le streaming temps réel, inutile pour des analyses historiques).



Architecture retenue : Sources (fichiers CSV et exports de base) → script d'anonymisation → Data Lake sur HDFS (zone brute) → traitement batch (Hive/Spark) → Data Warehouse Hive (zone exploitée) → Data Marts → Power BI.



Modèle logique en étoile (ou en constellation) :

- Tables de faits : Consultation, Hospitalisation, Décès, Satisfaction.
- Dimensions partagées : Temps (jour, mois, année), Patient pseudonymisé (sexe, tranche d'âge, département), Établissement (localisation, région), Diagnostic, Professionnel de santé.
- Chaque table de faits porte des mesures simples (nombre de consultations, durée de séjour, nombre de décès, note de satisfaction) et des clés vers les dimensions.

Tests de connexion : Vérification de l'accès aux services (HDFS, Hive) depuis la VM et depuis le poste qui héberge Power BI (pilote ODBC Hive/Impala).

3. Développement des scripts d'ingestion et chargement dans le Data Lake (2,5 h)

Ingestion : Les fichiers anonymisés sont copiés dans HDFS (hdfs dfs -put ou script Python/Sqoop selon la source), dans une zone « raw » organisée par source et par date de chargement.

/chu

├── raw/ ← données anonymisées, telles que reçues

│ ├── consultations/date_chargement=2026-10-20/

│ ├── hospitalisations/date_chargement=2026-10-20/

│ ├── deces/date_chargement=2026-10-20/

│ └── satisfaction/date_chargement=2026-10-20/

├── dwh/ ← tables de faits et dimensions (Parquet)

│ ├── fait_hospitalisation/annee=2025/mois=01/ …

│ └── dim_patient/ …

├── marts/ ← sous-ensembles par thème

│ ├── activite/

│ ├── mortalite/

│ └── satisfaction/

└── logs/ ← journal des chargements (traçabilité)

Contrôles qualité : Vérification du nombre de lignes avant et après chargement, détection des doublons, des valeurs manquantes et des formats incohérents (dates, codes diagnostics).

Traçabilité : Chaque chargement est journalisé (date, fichier source, nombre de lignes, opérateur) pour garder l'historique des traitements, comme l'exige le RGPD.

4. Création des tables physiques et alimentation du Data Warehouse et des Data Marts (3 h)

Modèle physique : Création des tables Hive à partir du modèle logique, avec des types adaptés (INT, STRING, DATE, DECIMAL).

```
CREATE TABLE fait_hospitalisation (
  id_temps           INT,
  id_patient_hash    STRING,
  id_etab            INT,
  code_diag          STRING,
  duree_sejour       INT,
  nb_hospitalisations INT
)
PARTITIONED BY (annee INT, mois INT)
STORED AS PARQUET;
```

Format et partitionnement : Stockage en Parquet (format colonnaire et compressé, donc des lectures plus rapides et moins d'espace disque). Partitionnement des tables de faits par année et par mois, et éventuellement bucketing par établissement.

Alimentation (ETL) : Scripts HiveQL ou Spark qui transforment les données brutes du Data Lake (nettoyage, jointures, calcul des clés de dimensions) et alimentent les tables de faits et de dimensions.

Data Marts : Création de sous-ensembles par thème d'analyse (activité hospitalière, mortalité, satisfaction), qui ne contiennent que les colonnes nécessaires à chaque usage.

5. Création des vues OLAP, agrégations et tests de performance (2 h)

Vues et agrégats : Création de vues pré-agrégées sur les axes d'analyse les plus demandés (par exemple le nombre d'hospitalisations par établissement et par mois, ou le taux de décès par région et par année).

Tests de performance : Comparaison des temps de réponse entre trois cas : le fichier brut, une table non partitionnée et une table en Parquet partitionnée. Le résultat sert d'argument pour justifier l'architecture devant le CHU.

Requête testée	Fichier brut	Table texte non partitionnée	Table Parquet partitionnée
Nombre d'hospitalisations par établissement et par mois	… s	… s	… s
Taux de décès par région et par année	… s	… s	… s
Note de satisfaction moyenne par établissement	… s	… s	… s

Optimisation : Calcul des statistiques des tables (COMPUTE STATS sous Impala) et ajustement des partitions si certaines requêtes restent trop lentes.

6. Connexion BI, indicateurs et contrôle RGPD (2 h)

Connexion Power BI : Connexion à Hive ou Impala via ODBC. Le mode DirectQuery permet d'interroger les données sans les recopier, tandis que le mode Import convient mieux aux agrégats légers.

Tableaux de bord : Création des indicateurs du cahier des charges (taux de consultation, taux d'hospitalisation, durée moyenne de séjour, mortalité, satisfaction). Ils sont filtrables par période, établissement, tranche d'âge, sexe et diagnostic.

Besoin du CHU	Indicateur	Data Mart	Visualisation Power BI
Suivre l'activité	Nombre de consultations par période et par établissement	Activité	Courbe dans le temps
Mesurer l'hospitalisation	Taux d'hospitalisation, durée moyenne de séjour	Activité	Histogramme par diagnostic
Suivre la mortalité	Nombre de décès par région et par année	Mortalité	Carte par région
Évaluer la qualité	Note moyenne de satisfaction	Satisfaction	Jauge et classement

Contrôle RGPD :

- Vérifier qu'aucune donnée directement identifiante n'apparaît dans les rapports.
- Masquer les cellules dont l'effectif est trop faible (par exemple moins de 5 ou 10 patients), car elles permettraient une réidentification.
- Restreindre les accès par profil (sécurité au niveau des lignes dans Power BI, droits sur les tables Hive).

7. Recette fonctionnelle et documentation technique (1 h)

Recette : Pour chaque analyse du cahier des charges, on vérifie que le tableau de bord donne un résultat cohérent, en le recoupant avec une requête manuelle sur un échantillon.

Documentation : Schéma d'architecture final, modèle logique et modèle physique, procédure d'ingestion et d'anonymisation, dictionnaire des données, gestion des accès, et limites connues de la maquette.

8. Préparation et présentation de la maquette au CHU (1,5 h)

Scénario de démonstration : Le parcours suit la donnée de bout en bout : donnée brute sécurisée, anonymisation, Data Lake, entrepôt, puis tableau de bord.

Messages clés : La faisabilité est démontrée, les gains de performance sont mesurés, la conformité RGPD est assurée et la solution peut passer à l'échelle.

Perspectives : Pour passer en production, il faudra un hébergeur certifié HDS, une gestion fine des identités (Kerberos, Ranger), une automatisation des flux (orchestrateur) et une montée en charge sur un vrai cluster.

7. Conclusion

Le travail d'Archie était une bonne base de départ (veille, environnement Cloudera, première vision de l'architecture), mais il n'était pas exploitable en l'état. L'architecture était trop complexe pour une maquette, mal reliée à l'environnement réellement installé, et les données de santé étaient stockées sans aucune sécurité. Plutôt que de tout reprendre ou de tout réutiliser tel quel, nous avons trié l'existant pour ne garder que ce qui servait le besoin du CHU.

La solution retenue repose sur une chaîne simple et maîtrisable. Les données sont d'abord anonymisées, puis stockées dans un Data Lake sur HDFS, structurées dans un entrepôt en étoile au format Parquet partitionné, déclinées en Data Marts par thème et restituées dans Power BI. Ce découpage répond au problème de performance, puisque les données ne sont plus interrogées dans leur forme brute, et au besoin d'analyse, puisque le modèle dimensionnel est construit à partir des indicateurs du cahier des charges.

Enfin, ce projet montre qu'un projet sur des données de santé ne se résume pas à un défi technique. La sécurité et la conformité RGPD (anonymisation avant ingestion, chiffrement, contrôle d'accès, traçabilité, seuils de diffusion) ont été traitées dès la première étape, et non ajoutées à la fin. La maquette démontre la faisabilité en 3 semaines. Le passage en production demandera en plus un hébergement certifié HDS et une gouvernance des données plus complète.

Prosit 2 : **Transformers**

Rôles :

- Animateur : Romain
- Scribe : Enzo
- Secrétaire : Thaïs
- Gestionnaire du temps : Lisa

1. Contexte

Nous avons maintenant une vision plus précise de l'architecture d'entreposage de données à mettre en place pour le projet CHU. L'analyse des données a montré que les sources sont hétérogènes : certaines sont structurées, d'autres semi-structurées ou non structurées. Archie l'avait anticipé en prévoyant dans son architecture un référentiel de stockage intermédiaire, le Data Lake. Celui-ci centralise toutes les données en une source unique, quel que soit leur format, tout en garantissant leur confidentialité.

Il faut maintenant construire l'entrepôt de données qui s'appuiera sur ce référentiel. Il doit supporter le volume, la vélocité et la variété des données (les 3V), ainsi que les charges de travail analytiques. Cela repose sur une modélisation dimensionnelle, différente de la modélisation relationnelle classique, et pensée à partir des analyses attendues par les utilisateurs.

L'intégration des données dans ce modèle pose plusieurs difficultés. Certaines informations nécessaires n'existent pas dans les sources : par exemple, la région n'apparaît pas alors que le CHU veut des analyses par région. D'autres informations sont réparties entre plusieurs sources différentes. Enfin, le format des dates fournies ne correspond pas à celui attendu par Hive. Les données devront donc être transformées, ce qui est le rôle de l'ETL.

2. Mots inconnus

Mot inconnu	Définition
Entreposage de données	Ensemble des techniques qui collectent, intègrent et stockent des données de plusieurs sources dans un entrepôt, pour l'aide à la décision.
Système décisionnel (BI)	Système qui transforme les données en informations utiles pour piloter et décider (tableaux de bord, indicateurs).
Données structurées	Données organisées selon un schéma fixe en lignes et colonnes (tables SQL, CSV).
Données semi-structurées	Données organisées par balises ou clés, sans schéma rigide (JSON, XML).
Données non structurées	Données sans organisation prédéfinie (texte libre, PDF, images, comptes rendus).
Référentiel de stockage intermédiaire	Espace centralisé (Data Lake) où les données brutes sont stockées dans leur format d'origine. Le schéma est appliqué à la lecture (schema-on-read).
3V	Volume (quantité), Vélocité (vitesse de production et de traitement), Variété (diversité des formats).
Charge de travail analytique	Ensemble des requêtes d'analyse lancées sur l'entrepôt (agrégations, filtres sur de gros volumes). Il s'oppose à la charge transactionnelle (OLTP).
Modélisation dimensionnelle	Conception d'un entrepôt en faits (ce qu'on mesure) et dimensions (selon quoi on analyse), à partir des analyses attendues.
Table de faits	Table centrale qui enregistre les événements à analyser : mesures numériques et clés vers les dimensions.
Dimension	Table qui décrit le contexte d'un fait (quand, où, qui, quoi) et sert d'axe d'analyse.

Mot inconnu	Définition
Schéma en étoile	Une table de faits reliée directement à ses dimensions.
Schéma en flocon	Les dimensions sont découpées en sous-tables (ex. Ville → Département → Région).
Schéma en constellation	Plusieurs tables de faits partagent les mêmes dimensions.
Intégration de données	Processus qui rassemble des données de sources différentes dans un modèle cible commun et cohérent.
ETL (Extract, Transform, Load)	Extraction des données, transformation (nettoyage, conversion, enrichissement), puis chargement dans l'entrepôt.
ELT (Extract, Load, Transform)	Chargement des données brutes d'abord, puis transformation dans l'outil cible. Adapté au Big Data car il utilise la puissance du cluster.
ETLT (Extract, Transform, Load, Transform)	Variante hybride : une transformation légère avant le chargement (par exemple la pseudonymisation RGPD), puis les transformations métier après le chargement, dans le cluster.
Enrichissement de donnée	Ajout d'informations absentes des sources grâce à un référentiel externe (ex. table département → région).
Hive	Outil Hadoop qui permet d'interroger les données de HDFS avec un langage proche du SQL (HiveQL).
Format de date Hive	Hive attend yyyy-MM-dd (et yyyy-MM-dd HH:mm:ss pour les horodatages). Une date en dd/MM/yyyy doit être convertie.

3. Problématique

Comment transformer et charger efficacement des données hétérogènes dans un entrepôt de données décisionnel, en respectant le RGPD, et faut-il transformer avant ou après le chargement ?

4. Contraintes

Contraintes sur les données

- **Sources hétérogènes** : données structurées, semi-structurées et non structurées.
- **Volume et historique** : plusieurs années de données issues des SI opérationnels d'HCG (les 3V : volume, variété, vélocité).
- **Données incomplètes** : certaines infos utiles aux analyses n'existent pas dans les sources, par exemple la région.
- **Données dispersées** : des informations nécessaires sont réparties entre plusieurs sources et doivent être croisées.
- **Formats incompatibles** : le format de date des données ne correspond pas à celui attendu par Hive.

Contraintes réglementaires

- **RGPD** : ce sont des données personnelles de santé, donc il faut les anonymiser ou les pseudonymiser.
- **HDS** : l'infrastructure doit respecter la certification Hébergeur de Données de Santé.
- **Gouvernance des données** : il faut une gestion rigoureuse des accès, de la traçabilité et de la qualité.

Contraintes techniques

- **Modèle dimensionnel imposé** : les données doivent être intégrées dans un schéma en étoile ou en flocon (faits et dimensions), pensé selon les analyses attendues par les utilisateurs.

- **Architecture existante** : il faut partir de celle d'Archie, avec un Data Lake comme référentiel intermédiaire et un ETL déjà positionné.
- **Choix ETL ou ELT** : il faut justifier si les transformations se font avant ou après le chargement.

Contraintes projet

- **Délais serrés** : très peu de marge dans le planning, il faut produire rapidement le premier livrable.

5. Plan d’action [8h]

1. Lire les ressources et définir les mots-clés (ETL, ELT, Data Lake, schémas en étoile et constellation, Hive) : 1 h
2. Comparer ETL et ELT et trancher sur la position de l’ETL chez Archie : 45 min
3. Concevoir le modèle dimensionnel (constellation : faits, dimensions, grain) : 1 h
4. Analyser les sources et lister les transformations nécessaires (dates, région, jointures, pseudonymisation) : 1 h 15
5. Adapter le pipeline existant (dates, région, pseudonymisation) et charger dans l’entrepôt (Iceberg/Trino puis ClickHouse) : 2 h
6. Tester avec quelques requêtes de contrôle : 30 min
7. Rédiger le livrable et préparer le retour : 1 h 30

6. Réalisation

6.1 ETL ou ELT : position de l'ETL chez Archie

Nous retenons l'ELT : les données sont chargées brutes dans le Data Lake, puis transformées dans le cluster avant d'alimenter l'entrepôt. Dans le schéma d'Archie, la brique Anonymisation (ETL) » est placée entre les sources et le Data Lake : les données y sont transformées avant d'être stockées. Ce n'est pas adapté à un Data Lake, qui conserve les données brutes et applique le schéma à la lecture. La modification consiste à déplacer cette transformation après le chargement : le Data Lake reçoit les données brutes (EL), puis le traitement batch réalise toutes les transformations (T), dont la pseudonymisation.

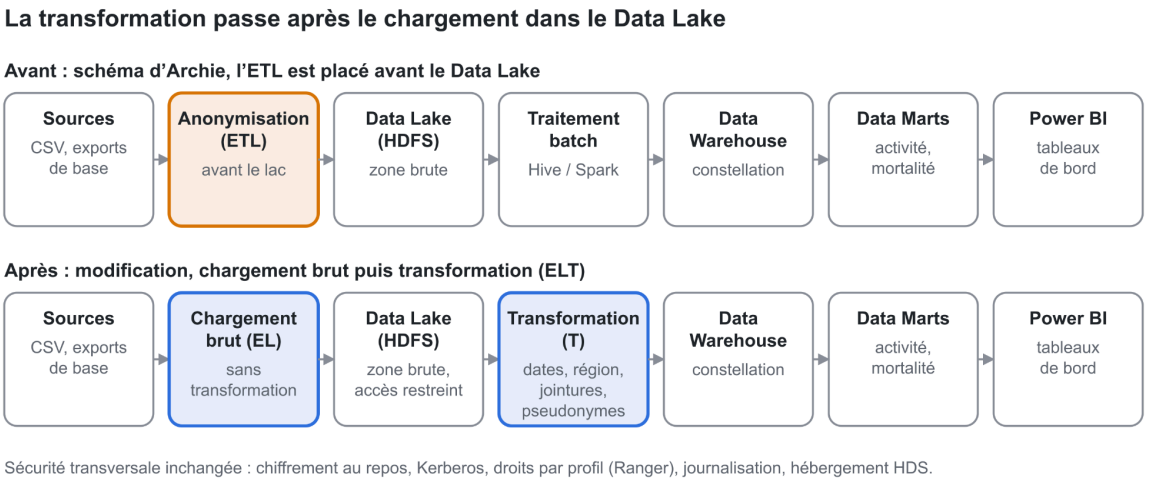


Figure 1 : modification du schéma d'Archie, de l'ETL vers l'ELT

Critère	ETL	ELT
Moment de la transformation	Avant le chargement, sur un serveur ETL dédié	Après le chargement, dans le cluster cible
Données brutes conservées	Non, seule la version transformée est chargée	Oui, rejouables si une règle change
Volume supporté	Limité par le serveur ETL	Puissance du cluster (Spark, Hive)
Variété des formats	Schéma imposé à l'écriture	Schema-on-read, tous formats acceptés

Critère	ETL	ELT
RGPD	Données nominatives jamais chargées	Zone brute à protéger (accès restreint, chiffrement)
Cas d'usage	Petits volumes, schéma stable	Big Data, besoins qui évoluent

Arguments du choix :

- **Les 3V** : le volume et la variété imposent de transformer là où se trouve la puissance de calcul.
- **Agilité** : les besoins du cahier des charges évoluent ; garder le brut permet de recalculer sans ré-extraire les SI du CHU.
- **Délais** : on charge vite, puis on affine les transformations couche par couche.
- **Limite** : la zone brute contient des données de santé nominatives. Elle est donc réservée aux administrateurs, et la pseudonymisation se fait dès la première transformation.

Pipeline :

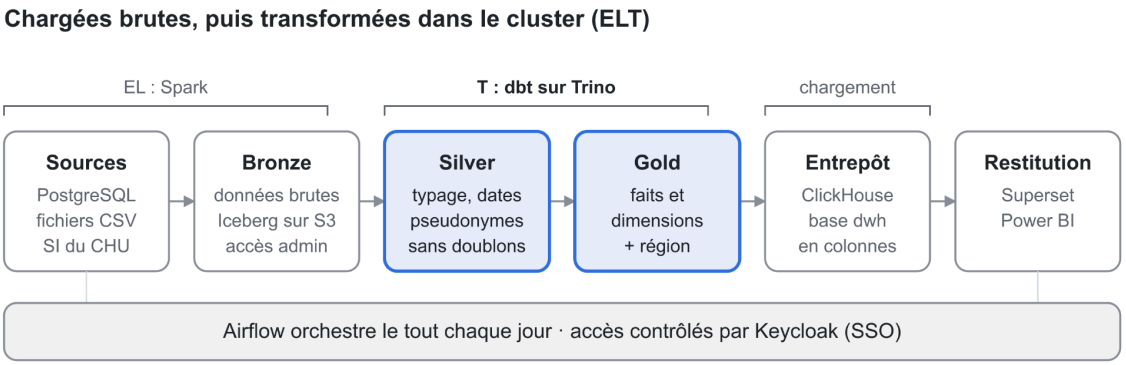


Figure 2 : pipeline ELT du projet CHU

6.2 Modèle dimensionnel : schéma en constellation

Le modèle cible est une constellation : quatre tables de faits partagent les dimensions Patient, Temps, Diagnostic et Établissement (avec la région).

Table de faits	Grain (une ligne =)	Mesures	Dimensions
Consultation	une consultation	nombre de consultations	Patient, Professionnel, Diagnostic, Temps, Mutuelle
Hospitalisation	un séjour	nombre de séjours, durée (jours)	Patient, Établissement, Diagnostic, Temps (entrée, sortie)
Décès	une région, une année	nombre de décès	Localisation (région, département), Temps
Satisfaction	un établissement, une année	score global	Établissement, Temps

La région est un attribut de la dimension Établissement (dénormalisée en étoile) plutôt qu'une sous-table en flocon : on évite une jointure à chaque requête.

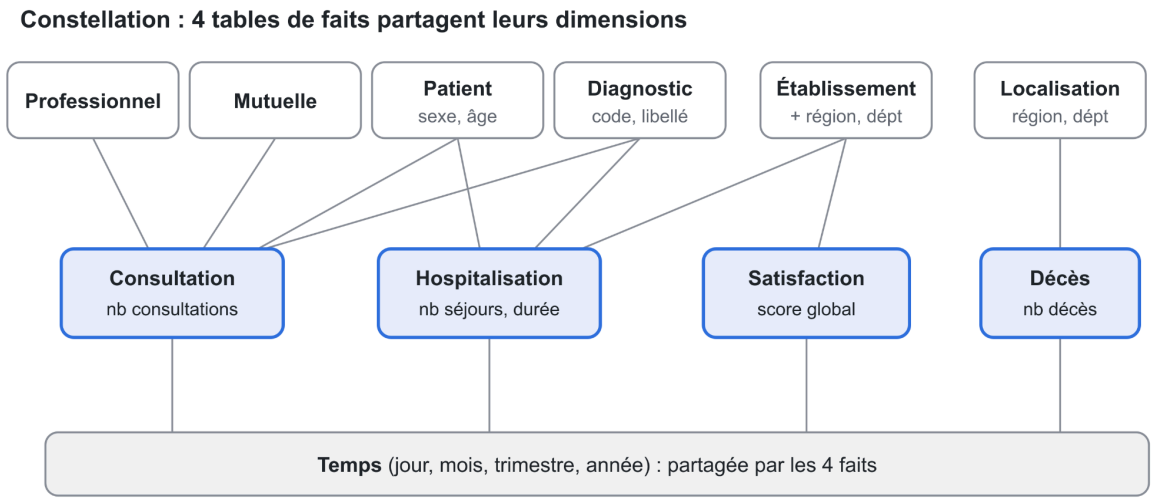


Figure 3 : schéma en constellation du modèle cible

6.3 Transformations nécessaires

Problème	Transformation	Exemple
Format de date	Conversion dd/MM/yyyy vers yyyy-MM-dd	to_date(from_unixtime(unix_timestamp(date_entree, 'dd/MM/yyyy'))) en HiveQL

Problème	Transformation	Exemple
Région absente	Enrichissement : code postal, puis département, puis région via un référentiel INSEE	2 premiers chiffres du code postal (3 pour les DOM 97x, 2A/2B pour la Corse), puis jointure sur la table département → région
Informations dispersées	Jointures entre sources sur une clé commune	hospitalisations (CSV) + établissements (CSV) sur identifiant_organisation ; consultations (PostgreSQL) + professionnels
Données personnelles	Pseudonymisation et minimisation	hash SHA-256 salé de l'identifiant patient, suppression nom, prénom, adresse, âge en tranches
Doublons	Dédoublonnage	une ligne par établissement (row_number() par organisation)
Types incohérents	Typage sûr, valeurs invalides mises à NULL	try_cast(jour_hospitalisation as integer)
Dimension Temps	Génération du calendrier	année, trimestre, mois, jour de la semaine à partir des dates des faits

6.4 Pipeline mis en œuvre

Le pipeline suit une architecture en médaillon, exécutée chaque jour par Airflow (DAG lakehouse_pipeline).

1. **Extraction et chargement (EL)** : un job Spark copie les 13 tables PostgreSQL et les fichiers CSV (décès, établissements, professionnels, hospitalisations, satisfaction 2019) dans la couche bronze, en Iceberg sur stockage S3.
2. **Transformation silver** : dbt, exécuté via Trino, type, nettoie, dédoublonne et convertit les dates (modèles stg_*).
3. **Transformation gold** : dbt construit les faits, les dimensions et les agrégats (fact_consultation, fact_hospitalisation, dim_patient, dim_date, agg_deces, agg_satisfaction_region…).
4. **Chargement de l'entrepôt** : les tables gold sont copiées dans ClickHouse (base dwh), stockage en colonnes adapté aux charges analytiques.
5. **Restitution** : Superset (et Power BI) interroge l'entrepôt.

Trino et Iceberg jouent ici le rôle que Hive tient dans l'architecture d'Archie : requêtes SQL sur des fichiers du Data Lake. Les règles de transformation restent les mêmes, seule la syntaxe change (date_parse(date_entree, '%d/%m/%Y') en Trino). L'accès est contrôlé par SSO Keycloak, avec un groupe administrateur et un groupe lecture seule.

6.5 Requêtes de contrôle

- **Volumétrie** : même nombre de lignes entre bronze et silver, hors doublons supprimés.
- **Dates** : aucune date NULL après conversion, sinon le format source n'est pas celui attendu.
- **Région** : aucun établissement sans région après enrichissement.
- **Intégrité** : chaque clé étrangère d'une table de faits existe dans sa dimension (tests dbt not_null, unique, relationships).

```
-- dates non converties

select count(*) from silver.stg_hospitalisation where
   date_entree is null;



-- séjours sans région

select count(*) from gold.fact_hospitalisation where region is
   null;



-- durée moyenne de séjour par région et par année

select region, year(date_entree) as annee, avg(duree_jours) as
   duree_moy

from gold.fact_hospitalisation

group by 1, 2

order by 1, 2;
```

7. Conclusion

Pour répondre à la problématique, nous chargeons d'abord les données brutes dans le Data Lake, puis nous les transformons dans le cluster : c'est une approche ELT. La modification apportée au schéma d'Archie déplace la transformation, placée avant le Data Lake, après le chargement des données brutes.

Les transformations règlent les trois difficultés du contexte : conversion des dates au format attendu par Hive, enrichissement de la région via le code postal et un référentiel, et jointures entre sources. La pseudonymisation dès la couche silver et le contrôle des accès répondent au RGPD. Le modèle en constellation permet d'analyser consultations, hospitalisations, décès et satisfaction selon des dimensions communes.

Prochaines étapes : ajouter la table de référence département → région, intégrer les faits Décès et Satisfaction au modèle, et mesurer les temps de réponse des requêtes sur l'entrepôt.

Prosit 3 : Bee patient

Rôles :

- Animateur : Lisa
- Scribe : Romain
- Secrétaire : Enzo
- Gestionnaire du temps : Thaïs

1. Contexte

Lors d'une réunion avec un panel d'utilisateurs composé de médecins, d'infirmiers, de chercheurs et d'administrateurs, le besoin d'analyse du CHU a été revalidé sans modification. La modélisation conceptuelle reste donc inchangée, et l'ETL ne sera modifié que pour les calculs d'agrégation. L'équipe lance maintenant l'implémentation du modèle physique de l'entrepôt sous ClickHouse, le remplissage des tables et l'écriture des requêtes d'analyse. Deux contraintes ont été fixées. D'une part, les données sources et les nouvelles analyses seront mises à jour tous les 3 mois. D'autre part, chaque analyse doit être produite en moins de 2 minutes, alors que les requêtes portent sur des tables de plusieurs millions de lignes avec plus de 3 jointures. Il reste 3 jours pour livrer les analyses.

2. Mots inconnus

Mot inconnu	Définition
MapReduce	Modèle de programmation parallèle en deux phases. La phase *Map* découpe et transforme les données en paires clé-valeur, puis la phase *Reduce* agrège les résultats par clé.

Mot inconnu	Définition
SGBD NoSQL	Système de gestion de bases de données non relationnel, conçu pour la scalabilité horizontale. On distingue quatre familles : clé-valeur, document, colonne et graphe.
Modèle physique	Traduction du modèle logique en structures concrètes de stockage (tables, types, formats de fichiers, partitions, index).
Partitionnement	Découpage d'une table en sous-répertoires selon la valeur d'une colonne, par exemple l'année ou l'établissement. Une requête filtrée sur cette colonne ne lit alors que les partitions concernées.
Bucketing	Répartition des données d'une table en un nombre fixe de fichiers, appelés buckets, selon le hachage d'une colonne. Il accélère les jointures et l'échantillonnage sur cette colonne.
Vue matérialisée	Résultat d'une requête stocké physiquement et rafraîchissable. Elle évite de recalculer des jointures ou agrégations coûteuses à chaque interrogation.
Clé de tri (ORDER BY)	Ordre de tri des données sur disque dans ClickHouse, qui sert d'index pour accélérer les filtres.
Agrégation	Calcul de synthèse sur un ensemble de lignes (somme, moyenne, comptage…), souvent réalisé avec un GROUP BY.
Jointure	Opération qui combine les lignes de plusieurs tables à partir d'une clé commune. Elle est coûteuse sur de gros volumes.
Effet de bord	Conséquence indirecte et non souhaitée d'une action. Ici, c'était le risque que la réunion remette en cause le besoin.
BASE	Approche de nombreuses bases NoSQL, qui privilégie la disponibilité, avec une cohérence obtenue à terme plutôt qu'immédiatement.

Mot inconnu	Définition
Théorème CAP	Un système distribué ne peut garantir à la fois que deux des trois propriétés suivantes : Cohérence, Disponibilité (*Availability*) et tolérance au partitionnement réseau (*Partition tolerance*).
MergeTree	Moteur de table principal de ClickHouse, qui stocke les données en colonnes, triées et fusionnées en arrière-plan.
ACID	Désigne les quatre propriétés qui garantissent la fiabilité des transactions dans une base de données relationnelle (SGBDR) : **Atomicité** : une transaction s'exécute entièrement ou pas du tout. Si une étape échoue, tout est annulé. **Cohérence** : la transaction fait passer la base d'un état valide à un autre état valide, en respectant les contraintes (clés, intégrité…). **Isolation** : les transactions exécutées en même temps ne se perturbent pas, comme si elles s'exécutaient l'une après l'autre. **Durabilité** : une fois validée, une transaction est conservée, même en cas de panne.

3. Problématique

Comment optimiser le modèle physique de l'entrepôt ClickHouse pour fournir aux praticiens et à la direction les analyses demandées en moins de 2 minutes, tout en intégrant facilement les nouvelles analyses définies tous les 3 mois ?

4. Contraintes

Liées au secteur hospitalier

- **Confidentialité des données de santé** : ce sont des données sensibles soumises au RGPD, qui doivent être pseudonymisées ou anonymisées et dont l'accès doit être contrôlé.
- **Hébergement** : en France, les données de santé doivent être hébergées chez un hébergeur certifié HDS (Hébergeur de Données de Santé).
- **Utilisateurs non techniques** : médecins et administrateurs doivent obtenir des résultats lisibles et exploitables.

5. Plan d'action [7h]

1. Lire les ressources et définir les mots-clés (NoSQL, MapReduce, partitionnement, bucketing, vues matérialisées) : 0,75 h(documentation)
2. Comparer SGBDR et NoSQL (types de bases, ACID vs BASE, théorème CAP) et situer ClickHouse : 0,5 h (documentation et restitution)
3. Analyser et repérer les filtres et jointures fréquents, les requêtes demandées et choisir les clés de partition et de tri : 1,5 h (réalisation)
4. Créer les tables ClickHouse en MergeTree (PARTITION BY, ORDER BY): 0,75 h (réalisation avec l'interface)
5. Adapter les modèles dbt pour les agrégations et charger les données dans ClickHouse: 1 h (réalisation avec l'interface)
6. Écrire les requêtes, mesurer les temps, puis optimiser les vues matérialisées pour passer sous 2 min : 1,5 h (réalisation avec l'interface)
7. Rédiger le livrable et préparer le retour : 1 h (restitution)

6. Réalisation

6.1 Comparaison SGBDR / NoSQL et place de ClickHouse

Les SGBDR (PostgreSQL, MySQL) rangent les données dans des tables avec un schéma fixe. Leurs transactions respectent ACID, donc les données restent toujours fiables. Les bases NoSQL (MongoDB, Cassandra) ont un schéma plus souple et se répartissent sur plusieurs serveurs. Elles suivent l'approche BASE : la base reste disponible, mais les données ne sont pas toujours à jour partout au même moment.

Le théorème CAP résume ce compromis : un système réparti ne peut pas tout garantir, il doit choisir entre cohérence et disponibilité.

ClickHouse est entre les deux. Il utilise le SQL et les jointures comme un SGBDR, mais il stocke les données par colonne : il ne lit que les colonnes utiles à la requête. Il n'est pas totalement ACID, mais ce n'est pas gênant pour nous, puisqu'on charge les données une fois par trimestre et qu'on fait surtout de la lecture.

6.2 Analyse des requêtes et choix des clés

Presque toutes les analyses demandées filtrent sur la période, l'établissement et le diagnostic, et font 3 à 4 jointures. On a donc choisi :

- **Partition par trimestre** : chaque mise à jour remplit une nouvelle partition. Une requête sur 2025 ne lit que les 4 partitions de 2025.
- **Tri par établissement, diagnostic puis date** : cette clé sert d'index et permet à ClickHouse de sauter les blocs inutiles.

ClickHouse n'a pas de bucketing (l'équivalent, le sharding, sert à répartir les données sur plusieurs serveurs, ce qui est inutile pour nous). Il n'utilise pas non plus MapReduce, mais il suit le même principe en exécutant chaque requête en parallèle.

6.3 Création des tables ClickHouse

Les tables utilisent le moteur MergeTree, qui stocke les données en colonnes, triées et fusionnées en arrière-plan.

```sql
CREATE TABLE fait_hospitalisation ( ... )

ENGINE = MergeTree

PARTITION BY toStartOfQuarter(date_entree)

ORDER BY (id_etablissement, id_diagnostic, date_entree);
```

On a choisi des types de colonnes légers et copié le sexe et la tranche d'âge du patient dans la table de faits, ce qui évite une jointure dans la plupart des requêtes.

6.4 Adaptation de dbt et chargement des données

Seuls les modèles dbt d'agrégation ont été modifiés. Ils sont en mode incrémental : à chaque trimestre, on ajoute seulement les nouvelles données. Spark dépose les données nettoyées sur S3 au format Parquet, puis ClickHouse les charge directement dans la bonne partition.

6.5 Écriture des requêtes, mesures et optimisation

On a écrit les requêtes d'analyse, comme le nombre d'hospitalisations par diagnostic et par établissement, puis mesuré leur temps d'exécution. Dans les jointures, la grosse table de faits est placée à gauche et les petites dimensions à droite, car ClickHouse charge la table de droite en mémoire.

Pour les requêtes trop lentes, on a créé des vues matérialisées. Elles stockent un résultat déjà calculé et se mettent à jour à chaque chargement, ce qui permet de passer sous les 2 minutes. Pour une nouvelle analyse demandée dans 3 mois, il suffit d'ajouter une nouvelle vue, sans modifier les tables existantes.

7. Conclusion

ClickHouse, avec son stockage en colonnes, convient bien à notre entrepôt. La partition par trimestre, la clé de tri et les vues matérialisées permettent de produire les analyses en moins de 2 minutes, même sur plusieurs millions de lignes. Le modèle s’adapte aussi facilement aux mises à jour trimestrielles : chaque chargement remplit une nouvelle partition, et une nouvelle analyse se traduit par une simple vue supplémentaire. Enfin, les données restent pseudonymisées, protégées par des accès selon le profil et hébergées chez un hébergeur certifié HDS.

Prosit 4 : Raconte-moi une histoire

Rôles :

- Animateur : Thaïs
- Scribe : Lisa
- Secrétaire : Romain
- Gestionnaire du temps : Enzo

1. Contexte

Après plusieurs semaines de travail, nous avons réussi à mettre en place notre architecture Big Data, à intégrer les différentes sources de données et à construire notre entrepôt de données décisionnel avec Hive.

Nous avons également réussi à exploiter les données grâce à des requêtes analytiques, tout en prenant en compte la contrainte de temps de traitement de 2 minutes fixée par les utilisateurs.

Notre maquette est désormais fonctionnelle et nous arrivons à obtenir les résultats des analyses demandées. Cependant, notre travail ne s'arrête pas là. Nous devons maintenant proposer une première version du tableau de bord, appelée “ Quick Win ”, afin de montrer concrètement aux utilisateurs les bénéfices de notre solution.

L'objectif est de permettre aux médecins, aux personnels infirmiers, aux chercheurs et aux administrateurs d'accéder facilement aux informations dont ils ont besoin pour prendre leurs décisions.

Pour cela, nous devons réfléchir à la manière de présenter les résultats, aux graphiques à utiliser et aux interactions à proposer. Nous envisageons notamment de créer des graphiques génériques et paramétrables avec des filtres, afin que chaque utilisateur puisse personnaliser ses analyses sans avoir besoin de développer plusieurs vues différentes.

Nous devons donc trouver une solution simple, efficace et adaptée aux besoins de chaque métier, tout en respectant les contraintes de temps et les ressources disponibles.

2. Mots inconnus

Mot inconnu	Définition
Quick Win	Première réalisation rapide qui apporte un résultat concret et visible aux utilisateurs.
KPI	Indicateur clé de performance permettant de mesurer et de suivre une activité.
Data Storytelling	Technique qui consiste à raconter une histoire à partir des données pour faciliter leur compréhension.
Visualisation dynamique	Graphiques qui évoluent automatiquement en fonction des données sélectionnées.

3. Problématique

Comment concevoir un tableau de bord interactif, simple et personnalisable permettant aux différents utilisateurs du CHU d'exploiter facilement les données disponibles, de visualiser leurs indicateurs clés et de faciliter leurs prises de décision, sans devoir développer plusieurs vues indépendantes ?

5. Plan d’action [7h]

1. Analyser les besoins des utilisateurs et identifier les KPI prioritaires.[2h]
2. Concevoir le tableau de bord en organisant les informations de manière claire et intuitive.[1,5h]
3. Mettre en place des graphiques interactifs et des filtres personnalisables (SuperSet).[1,5h]
4. Intégrer le Data Storytelling pour faciliter la compréhension des données.[1,5h]
5. Tester et présenter le Quick Win aux différents utilisateurs du CHU. [0,5h]

6. Réalisation

Étape 1 : Identification des besoins des utilisateurs

Nous avons déjà organisé une réunion avec un panel représentatif des utilisateurs du CHU. Nous allons donc reprendre les besoins qui ont été validés lors de cet échange.

Nous distinguons quatre grandes catégories d'utilisateurs :

Les médecins : ils ont besoin de consulter des informations permettant de suivre l'activité médicale et d'analyser les données relatives aux patients. Le suivi du patient.

Le personnel infirmier : il souhaite disposer d'indicateurs permettant de suivre l'activité des services et l'organisation des soins.

Les chercheurs : ils ont besoin de pouvoir exploiter les données pour réaliser des analyses et identifier des tendances.

Les administrateurs : ils souhaitent avoir une vision globale de l'activité des établissements afin de faciliter le pilotage et la gestion.

Nous devons donc concevoir une interface suffisamment flexible pour répondre à ces différents besoins, tout en conservant une présentation commune.

Étape 2 : Définition des KPI

Nous allons sélectionner les indicateurs les plus pertinents à partir des données déjà intégrées dans notre entrepôt Hive.

Nous pouvons notamment prévoir les indicateurs suivants, sous réserve de leur présence dans les données disponibles :

- Le nombre de patients pris en charge.
- Le nombre d'hospitalisations.
- La durée moyenne des séjours.

- Le nombre d'admissions et de sorties.
- La répartition de l'activité par établissement ou région.
- L'évolution de l'activité sur différentes périodes.

(Faire un tableau)

Ces indicateurs permettront aux utilisateurs d'avoir rapidement une vision globale de l'activité du CHU.

Nous veillerons également à ce que chaque KPI soit accompagné d'un intitulé clair et, si nécessaire, d'une comparaison avec une période précédente.

Étape 3 : Conception de l'interface (Superset)

Nous choisissons de développer un tableau de bord centralisé qui regroupe les principales informations sur une seule interface.

Cette organisation permettra aux utilisateurs de trouver rapidement les informations importantes sans devoir naviguer entre plusieurs pages.

Nous privilégierons également une présentation sobre, avec des couleurs cohérentes et des intitulés compréhensibles par tous les métiers.

Étape 4 : Mise en place des graphiques génériques et des filtres

Afin de gagner du temps et d'éviter de développer plusieurs vues indépendantes, nous choisissons de mettre en place des graphiques génériques et paramétrables.

Le principe est de créer une seule structure de visualisation capable d'afficher différentes informations selon les paramètres sélectionnés.

Nous prévoyons plusieurs types de graphiques :

Graphiques en courbes : pour observer l'évolution du nombre d'hospitalisations ou de patients sur une période.

Graphiques en barres : pour comparer l'activité entre plusieurs établissements ou services.

Graphiques circulaires : pour visualiser la répartition des patients selon différentes catégories.

Tableaux : pour consulter les données détaillées.

Nous ajoutons également plusieurs filtres interactifs :

- Un filtre par date ou période.
- Un filtre par établissement.
- Un filtre par région.
- Un filtre par service.
- Un filtre par catégorie d'analyse.

Lorsqu'un utilisateur sélectionne une région ou une période, les graphiques et les indicateurs concernés se mettent automatiquement à jour.

Cette solution permet donc à chaque utilisateur de personnaliser sa consultation sans que nous ayons besoin de créer une page différente pour chaque métier.

Étape 5 : Mise en place du Data Storytelling

Nous souhaitons aller plus loin qu'une simple présentation de chiffres. Notre objectif est de permettre aux utilisateurs de comprendre rapidement ce que signifient les données affichées.

Pour cela, nous organisons notre tableau de bord selon une logique de lecture en plusieurs étapes :

Observer : afficher les principaux indicateurs afin de comprendre rapidement la situation générale.

Comparer : permettre de comparer les résultats entre plusieurs établissements, services ou périodes.

Analyser : identifier les évolutions, les différences et les éventuelles anomalies.

Comprendre : mettre en évidence les éléments qui expliquent les résultats observés.

Décider : fournir les informations nécessaires pour aider les utilisateurs à prendre leurs décisions.

Par exemple, si un administrateur constate une augmentation du nombre d'hospitalisations, il pourra sélectionner une période et un établissement afin d'identifier les services concernés et de comparer leur évolution.

Le graphique ne présente donc pas uniquement un résultat : il permet à l'utilisateur de comprendre la situation et d'approfondir son analyse.

Étape 6 : Tests et présentation du Quick Win

Une fois notre tableau de bord développé, nous procéderons à plusieurs tests afin de vérifier son bon fonctionnement.

Nous vérifierons notamment :

- Que les données affichées correspondent bien aux résultats obtenus avec Hive.
- Que les filtres fonctionnent correctement.
- Que les graphiques se mettent à jour automatiquement.
- Que les informations sont compréhensibles par les différents métiers.
- Que les temps d'affichage restent acceptables.
- Que les utilisateurs peuvent accéder uniquement aux données auxquelles ils sont autorisés à accéder.

Nous organiserons ensuite une démonstration auprès d'un groupe représentatif d'utilisateurs.

L'objectif sera de leur montrer concrètement comment consulter les KPI, appliquer des filtres et réaliser leurs propres analyses.

Leurs retours nous permettront d'identifier les éventuelles améliorations à apporter avant le déploiement plus large de la solution.

Conclusion

Pour conclure, nous arrivons à la dernière étape de notre projet CHU. Après avoir travaillé sur l'architecture Big Data, l'intégration des données, la modélisation de notre entrepôt et l'exploitation des informations avec Hive, nous devons maintenant rendre les résultats accessibles et compréhensibles pour les utilisateurs.

La mise en place d'un tableau de bord interactif avec des KPI pertinents, des graphiques génériques et des filtres personnalisables nous permettra de répondre aux besoins des différents métiers sans multiplier les développements.

L'utilisation du Data Storytelling permettra également de donner du sens aux données en facilitant leur lecture, leur comparaison et leur interprétation.

Cette première version Quick Win sera essentielle pour démontrer la valeur ajoutée de notre solution, convaincre les utilisateurs de son intérêt et favoriser son adoption au sein du CHU.

Enfin, il faudra garder à l'esprit que les données utilisées sont particulièrement sensibles puisqu'elles concernent le domaine de la santé. Nous devons donc maintenir les règles de confidentialité, de sécurité et de gestion des droits d'accès tout au long de l'utilisation de notre outil.

Notre objectif final est donc de transformer les données que nous avons réussi à exploiter en véritables outils d'aide à la décision pour l'ensemble des professionnels du CHU.